

AI Lab: Informed Search Techniques in Python

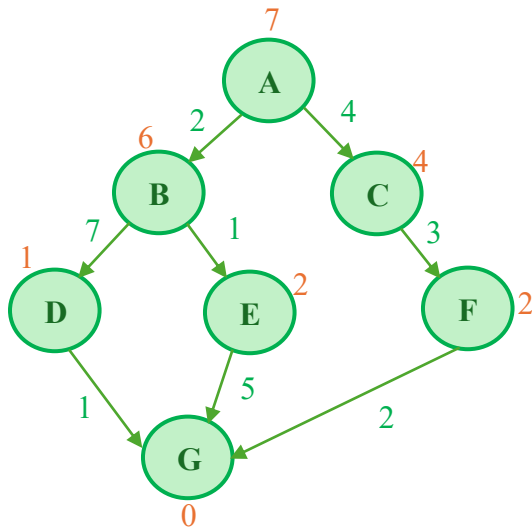
Lab Objectives:

1. Understand heuristic functions and their role in informed search strategies.
2. Implement **Greedy Best-First Search (GBFS)** and **A* Search** on a weighted graph.
3. Evaluate and compare search strategies in terms of path optimality and computational efficiency.

Task 1: Heuristic Function Evaluation

Problem Instance:

Weighted directed graph:



Node	Neighbors (Cost)
A	B(2), C(4)
B	D(7), E(1)
C	F(3)
D	G(1)
E	G(5)
F	G(2)
G	—

Goal: Node G

Heuristic Function: Estimated cost to G

Node	$h(n)$
A	7
B	6

C	4
D	1
E	2
F	2
G	0

Instructions:

1. Implement a Python function **h(n)** that returns the heuristic value.
2. Verify that the heuristic is **admissible** (never overestimates the true cost) and **consistent** (triangle inequality holds for each edge i.e. $h(n) \leq \text{cost}(n,m) + h(m)$ holds for all edges).
3. Tabulate $h(n)$ vs actual shortest path cost from each node to G.

Task 2: Greedy Best-First Search

Problem Instance:

Use the graph and heuristic from Task 1.

Instructions:

1. Implement **GBFS** using $f(n) = h(n)$ as the evaluation function.
2. Start at node **A**, goal node **G**.
3. Record:
 - Order of node expansions
 - Path found
 - Total path cost
4. Compare with **uninformed BFS** in terms of node expansions.

Deliverables:

- Python implementation of GBFS
- Node expansion sequence and path found
- Comparative analysis with BFS

Task 3: A* Search Algorithm

Problem Instance:

Same graph and heuristics as Task 1.

Instructions:

1. Implement **A* Search** using

$$f(n) = g(n) + h(n)$$

- $g(n)$ = cumulative path cost from start to node n
 - $h(n)$ = heuristic value
2. Start at node **A**, goal **G**.
 3. Record:
 - Nodes expanded
 - Optimal path and total cost
 4. Compare results with GBFS in terms of path optimality and node expansions.

Deliverables:

- Python implementation of A*
- Table comparing GBFS vs A*
- Analysis of why A* guarantees optimality while GBFS may not

Task 4: Weighted A*

Problem Instance:

Use the same graph. Modify A* evaluation function:

$$f(n) = g(n) + w \cdot h(n), w = 1.0, 1.5, 2.0$$

Instructions:

1. Run weighted A* for different values of w .
2. Observe changes in path found and number of nodes expanded.
3. Discuss trade-offs between optimality and efficiency.

Deliverables:

- Python implementation of weighted A*
- Table of results for each w
- Print the Solution Path and cost

Normal A*:

“Balance actual cost and estimated cost.”

Weighted A*:

“Trust heuristic more than actual cost.”

Does Weighted A* Guarantee Optimality?

- If $w = 1 \rightarrow$ Yes (normal A*)
- If $w > 1 \rightarrow$ No (may lose optimality)

But:

It often gives a near-optimal solution much faster.

Task 5: Informed Search on a Directed Graph with Multiple Goals

Problem Description

You are given a directed graph with nodes **A–L**, edge weights (costs), and heuristic values ($h(n)$) estimating the cost to the nearest goal.

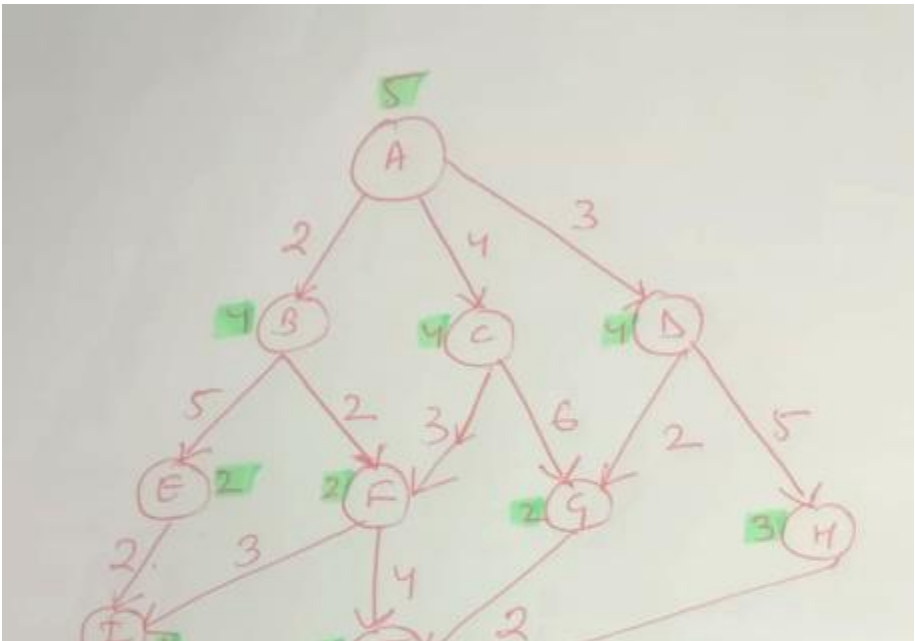
- **Start Node:** A
- **Goal Nodes:** K, L
- **Edges and Costs:**

From	To	Cost
A	B	2
A	C	4
A	D	3
B	E	5
B	F	2
C	F	3
C	G	6
D	G	2
D	H	5
E	I	2
F	I	3
F	J	4
G	J	2

H	J	3
I	K	1
J	K	2
J	L	3

Heuristic Values (h(n)): estimated cost to nearest goal:

Node	h(n)
A	5
B	4
C	4
D	4
E	2
F	2
G	2
H	3
I	1
J	1
K	0
L	0



Part 1: Greedy Best-First Search (GBFS)

1. Implement **GBFS** on the given graph using $f(n) = h(n)$ as the evaluation function.
2. Start at node **A**.
3. The search should stop upon reaching **any goal node** (K or L).
4. Record the following:
 - Order of node expansions
 - Path found (Solution Path)
 - Total path cost
5. Analyze which goal was reached and why GBFS chose that path.

Part 2: A* Search

1. Implement **A* Search** using $f(n) = g(n) + h(n)$ where $g(n)$ is the accumulated path cost.
2. Start at node **A**, and search for the **lowest-cost path to any goal node**.
3. Record the following:
 - Order of node expansions
 - Optimal path found
 - Total path cost
4. Compare the path found by A* with that of GBFS.

Part 3: Comparative Analysis

Compare **GBFS** and **A*** in terms of:

- Path optimality (cost to goal)
- Number of nodes expanded
- Efficiency of reaching a goal

Ungraded Tasks:

Task 6: Heuristic Function Evaluation on 8-Puzzle

Problem Instance:

Initial state (3×3 puzzle):

1	2	3
4	0	5
6	7	8

Goal state:

1	2	3
4	5	6
7	8	0

Instructions:

1. Implement the following heuristics:
 - **h1(n):** Number of misplaced tiles.
 - **h2(n):** Manhattan distance (sum of distances of each tile from its goal position).
2. Compute h1 and h2 for the initial state and two intermediate states (you may select two states resulting from a single legal move).
3. Verify which heuristics are admissible and consistent.

Deliverables:

- Python functions for h1 and h2.
- Table showing heuristic values for initial and two intermediate states.
- Short explanation on admissibility and consistency.

Task 7: Greedy Best-First Search (GBFS) on a Weighted Grid

Problem Instance:

Consider a 4×4 grid with the following cell costs:

S	1	4	7
2	X	1	3
3	2	2	1
4	3	1	G

- **S** = Start (0,0)
- **G** = Goal (3,3)
- **X** = Obstacle (cannot pass)

Heuristic: Manhattan distance to Goal.

Instructions:

1. Implement **GBFS** using the given grid and heuristic.
2. Show the order of nodes expanded.
3. Compute the path found and its total cost.
4. Compare the result with BFS (uninformed search).

Deliverables:

- Python implementation of GBFS.
- Printed node expansion order and final path with total cost.
- Short comparison with BFS.

***Note:** In all tasks, Visualize each graph using **networkx** Library.*

Submission Guidelines:

1. Prepare a **Word report** containing screenshots of the output for **all tasks**.
2. Ensure that each screenshot is **clearly placed under its respective task**.
3. **Download the .ipynb file** from Jupyter Notebook.
4. **Upload the Word report, and .ipynb** to GCR. **Do NOT upload a ZIP file.**

5. **Failure to follow the submission guidelines will result in a deduction of 50% of the total marks.**